Predicting Air Pollution with LSTM Neural Networks

A machine learning tutorial using the UCI Air Quality dataset

GitHub: https://github.com/mashrurR/Predicting-Air-Pollution-with-LSTM-Neural-Networks.git

This tutorial shows you how to build a two-layer Long Short-Term Memory (LSTM) neural network that forecasts the next hour's carbon monoxide (CO) concentration from a year's worth of hourly sensor readings collected in an Italian city. By the end you will have working code, an honest evaluation against a baseline, and enough understanding of LSTM to adapt the approach to your own environmental data.

1. Why LSTM for Pollution Data?

Pollution readings are not independent. Carbon monoxide measured at 8am depends partly on what happened at 7am, and on the general pattern of the morning commute. That dependence on history is called temporal structure, and standard feedforward networks ignore it — they treat each input as if it arrived from nowhere.

LSTM (Long Short-Term Memory) networks were designed specifically for sequences. Each LSTM cell maintains a hidden state and a cell state that it carries forward through time. Two learned gates — the forget gate and the input gate — control which information to keep from the past and which new information to let in. This lets the network hold onto patterns that span hours or days rather than forgetting them after a few steps.

Pollution is a natural fit because it shows at least two scales of structure: a daily cycle (rush-hour peaks, overnight lows) and weather-driven variation across days and seasons. LSTM can in principle learn both.

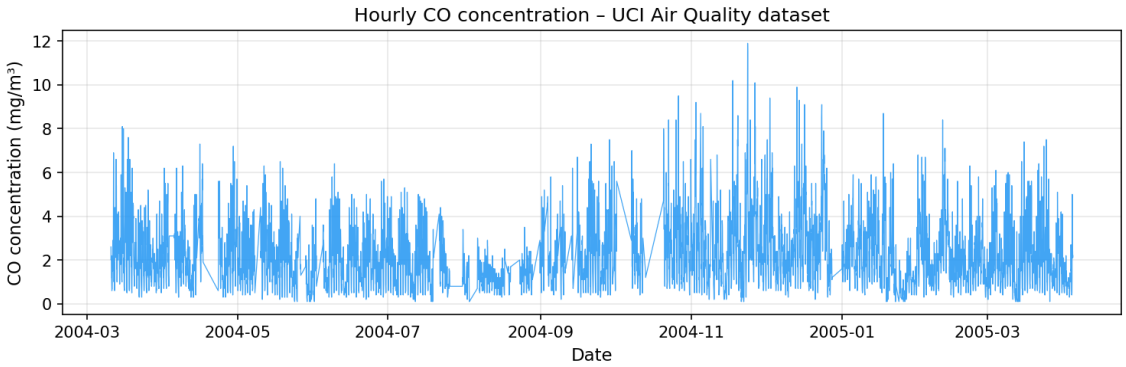

Figure 1. Hourly CO concentration over the full 13-month dataset. The visible seasonal variation and repeated daily patterns confirm that past values carry information about future ones.

2. The Dataset

The UCI Air Quality dataset was collected by De Vito et al. (2008) using a multi-sensor device placed at road level in a heavily trafficked area of an Italian city. It contains 9,358 hourly observations from March 2004 to April 2005 across 13 columns: ground-truth concentrations of

CO, benzene (C_6H_6), NOx, and NO_2 alongside sensor responses for each, plus temperature, relative humidity, and absolute humidity.

Two data quality issues need handling before any modelling. First, the file uses European formatting with semicolons as column separators and commas as decimal points. Second, the dataset uses -200 as a sentinel value for missing readings — these must be replaced with NaN before scaling or the model will learn that -200 is a valid concentration.

Loading and cleaning:

```
df = pd.read_csv('AirQuality.csv', sep=';', decimal=',')
df.replace(-200, np.nan, inplace=True)
df['Datetime'] = pd.to_datetime(df['Date'] + ' ' + df['Time'],
    format='%d/%m/%Y %H.%M.%S', errors='coerce')
df = df.dropna(subset=['Datetime', 'CO(GT)']).sort_values('Datetime')
```

After cleaning we have 7,674 complete hourly CO readings, spanning March 2004 to April 2005.

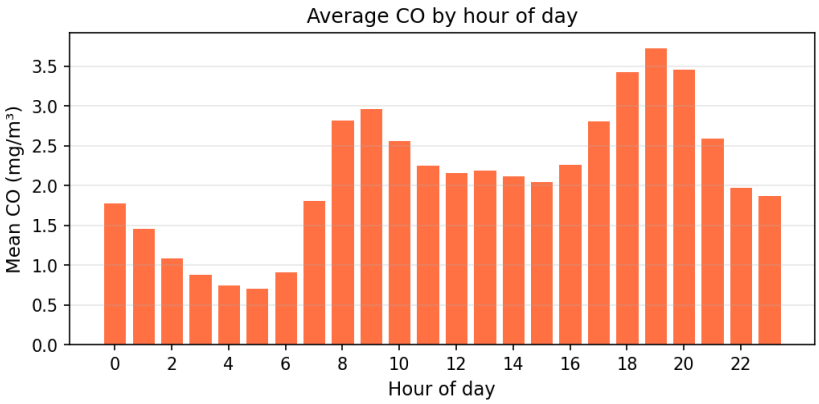

Figure 2. Mean CO by hour of day. The morning and evening commute peaks are visible. This predictable daily structure gives the LSTM something concrete to learn.

3. Preparing the Data for LSTM

Scaling

LSTM cells use sigmoid and tanh activations, both of which saturate outside the range roughly [-2, 2]. When inputs are large — CO values here go up to 11.9 mg/m³ — the gradient signal during training can vanish or explode. MinMaxScaler compresses everything to [0, 1], which keeps the network's internal arithmetic well-behaved.

The sliding window

We convert the 1D series into supervised input-output pairs using a 24-hour look-back window. For each hour t, the input X is the 24 previous CO readings and the target y is the reading at hour t. We chose 24 because it captures one full daily cycle, which Figure 2 shows is the dominant pattern in this data.

```
LOOK_BACK = 24
X, y = [], []
```

```
for i in range(LOOK_BACK, len(scaled)):
    X.append(scaled[i - LOOK_BACK:i, 0])
    y.append(scaled[i, 0])
X = np.array(X).reshape(-1, LOOK_BACK, 1) # shape: (samples, 24, 1)
```

Train/test split

We split chronologically: the first 80% of windows go into training, the last 20% into testing. Shuffling would be wrong here — it would allow future values to leak into the training set, making error metrics look artificially good. This gives us 6,120 training samples and 1,530 test samples.

4. Building the LSTM Model

The model is a two-layer stacked LSTM followed by a single output neuron. The first LSTM layer processes the 24-step sequence and passes its full output sequence to the second layer, which compresses it into a single fixed-length vector. Dropout (20%) between layers prevents the network from relying too heavily on any individual unit.

Layer	Type	Units	Notes
1	LSTM	64	return_sequences=True
2	Dropout	—	20%
3	LSTM	32	return_sequences=False
4	Dropout	—	20%
5	Dense	1	regression output

```
model = Sequential([LSTM(64, return_sequences=True, input_shape=(24, 1)),
Dropout(0.2), LSTM(32), Dropout(0.2), Dense(1)])
model.compile(optimizer='adam', loss='mse')
```

We use Adam as the optimiser and mean squared error (MSE) as the training loss. MSE penalises large errors more heavily than small ones, which is appropriate when we want to avoid badly wrong predictions.

Early stopping

We allow up to 30 training epochs but stop early if the validation loss does not improve for 5 consecutive epochs, restoring the weights from the best epoch. In practice the model converged after 14 epochs.

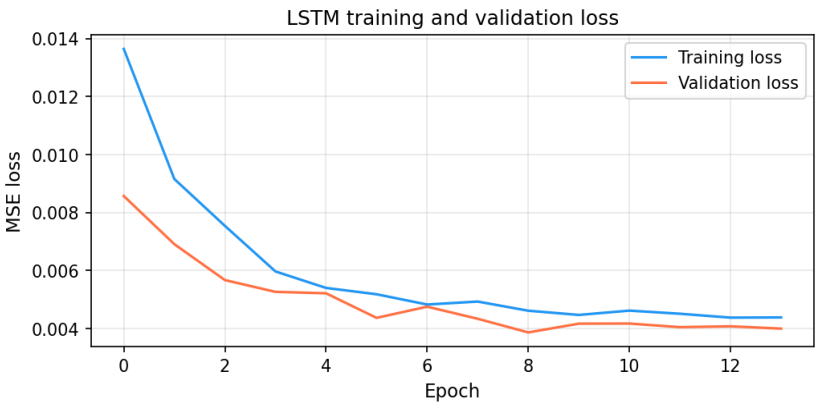

Figure 3. Training and validation loss over 14 epochs. Both curves decrease together without significant divergence, indicating the model is not overfitting.

5. Results

We compare the LSTM against a naive persistence baseline, which simply predicts that the next hour's CO equals the current hour's CO. This is the simplest possible time-series forecast, and any model that cannot beat it is not worth using.

Model	MAE (mg/m³)	RMSE (mg/m³)
LSTM (this tutorial)	0.4616	0.6843
Naive baseline	0.5148	0.7965

The LSTM reduces MAE by 10.3% and RMSE by 14.1% compared to the naive baseline. The RMSE improvement is larger, which tells us the model particularly helps on large errors — the spike events where the naive model simply repeats a lower previous value.

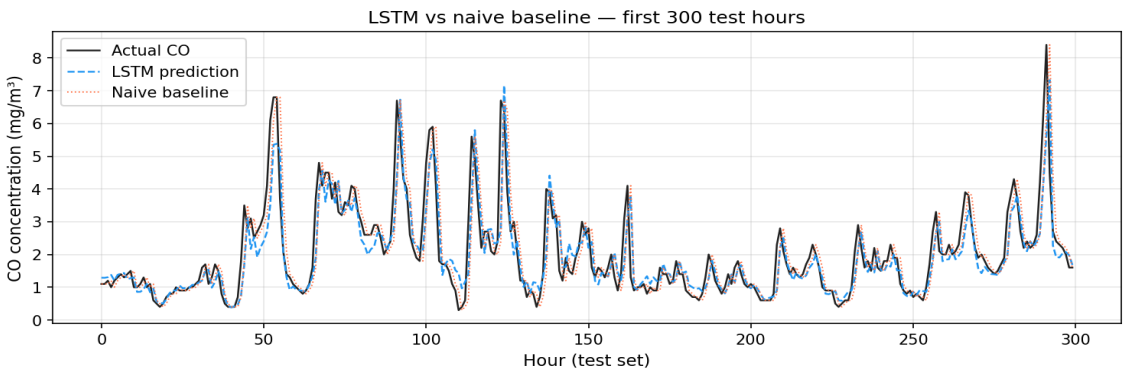

Figure 4. LSTM predictions (blue dashed) versus actual CO (black) and naive baseline (orange dotted) over the first 300 test hours. The LSTM tracks peaks more closely than the baseline.

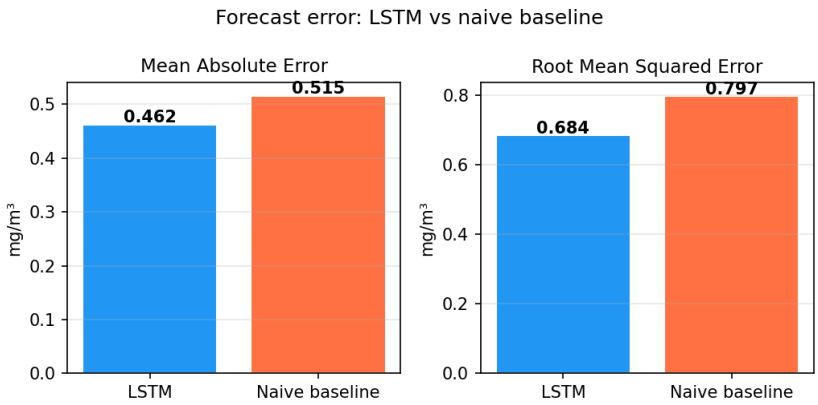

Figure 5. Error comparison. Lower is better. LSTM outperforms the naive baseline on both metrics.

6. Using This in Your Own Work

The code in this tutorial is written to be easy to adapt. If you want to forecast a different pollutant, change 'CO(GT)' in the preprocessing step to any other numeric column — 'NOx(GT)', 'C6H6(GT)' (benzene), 'NO2(GT)', and so on. If you have several sensors and want to use all of them as inputs, you can extend X to include multiple columns by changing the reshape step; the LSTM architecture itself does not need to change, only the input_shape parameter.

The look_back of 24 was chosen to capture one full day, but it is worth experimenting with other values. A look_back of 12 (half day) may be enough if your data has only short-range memory; 48 (two days) might help in datasets with stronger multi-day patterns. You can treat look_back as a hyperparameter and compare validation loss across values.

The same structure applies to any environmental time series — PM2.5, water temperature, noise levels — provided the data is collected at regular intervals and the target variable has some temporal structure. If it does not (if it is essentially random), LSTM will not help and a simpler model is more honest.

7. Ethical Considerations

Pollution forecasting can directly affect decisions about public health alerts, school closures, and industrial operations. A 10% error reduction sounds modest, but if the baseline model is already being used for those decisions, the improvement matters. At the same time, a model with a 0.46 mg/m³ MAE should not be reported as 'accurate' without context — on a day when CO peaks above 8 mg/m³, that same absolute error is a much smaller fraction of the reading.

Models trained on one city's sensor data do not automatically transfer to another city with different traffic patterns, road layouts, or weather. Any real deployment would need the model re-trained or fine-tuned on local data.

References

Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation, 9*(8), 1735-1780. https://doi.org/10.1162/neco.1997.9.8.1735

De Vito, S. et al. (2008). On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario. *Sensors and Actuators B: Chemical, 129*(2), 750-757. https://doi.org/10.1016/j.snb.2007.09.060

Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.

Brownlee, J. (2017). Time Series Forecasting with the Long Short-Term Memory Network in Python. Machine Learning Mastery. https://machinelearningmastery.com/time-series-forecasting-long-short-term-memory-network-python/

fedesoriano (2021). Air Quality Data Set. Kaggle. https://www.kaggle.com/datasets/fedesoriano/air-quality-data-set

UCI Machine Learning Repository: Air Quality Dataset. https://archive.ics.uci.edu/ml/datasets/Air+Quality